ITA03-MP MOBILE COMPUTING

Assignment 2 – CO2

Mobile Telecommunication Systems and Multiple Access Techniques (SDMA, TDMA, FDMA, CDMA, OFDMA)

Item	Details
Student	Rohit Kumar Sekar
Register No.	192312214
Nature	Individual Total marks: 50
Submission date	_____
Source code repository (GitHub)	_____ (upload the the code/ folder folder; README and fixed seeds included)
AI / reference tools disclosure	AI assistance (Claude) was used to draft the code and report text. Replace or confirm this statement with your own wording, and be ready to explain every line in the viva.

How this report is organised. Each question follows the Predict → Design → Implement → Measure → Analyse → Evaluate chain. All numbers in tables are produced by the Python programs in the code/ folder (fixed seeds, run python3 qN_*.py); raw data are in data/*.csv. Counted quantities (calls blocked, bits in error, bursts, messages, slot-ticks, bit/symbol) are the evidence; run time is not used.

Assumptions that are not standards values are stated where used (set-up times, FDMA overhead, SNR spread, lane width, user-defined thresholds). **Standards values used:** GSM 200 kHz carriers, 8 slots, 156.25-bit normal burst with 114 data bits and 270.833 kbit/s (ETSI/3GPP TS 45.001/45.002); OFDM subcarrier spacing 15 kHz (3GPP TS 36.211).

Q1 – Mobile Communication or Mobile Computing? (5 marks)

1.1 Working definitions

- **Wireless** describes the transmission medium (no cable in the path). **Mobile** describes the user or device moving while it is being used. The two are independent: a fixed-wireless router is wireless but not mobile; a laptop moved between offices and plugged into Ethernet is mobile (nomadic) but not wireless.
- **Mobile communication** is the link-level problem: radio access, multiple access, handover, coverage, call set-up, security of the air interface.
- **Mobile computing** is the application/system-level problem created by mobility: disconnected operation, data synchronisation, resource-poor devices, location/context awareness, adaptation of applications to variable connectivity.

1.2 My classification criteria (defined before analysing the cases)

Criterion	What it asks	Scale
K1 User mobility during use	Is the user static, moving only between sessions, or moving while using the device?	L = static; M = moves between sessions (nomadic); H = moves during use
K2 Wireless link in the path	Does the data path include a radio link, and how critical is it?	None / M (used only occasionally) / H (essential, continuous)
K3 Tolerance of disconnection	Must the application keep working when the network disappears?	L = not needed; M = occasionally; H = essential
K4 Location/context awareness	Does behaviour depend on where the user/device is or what it is doing?	L / M / H
K5 Device resource limits	Are battery, CPU, storage, screen or ruggedness real constraints?	L / M / H

Decision rules. *Class*: K1 = L and no radio → fixed-wired; K1 = L with radio → fixed-wireless; K1 = M → nomadic; K1 = H → mobile. *Main problem*: (i) if there is no radio link, it cannot be a mobile-communication problem; (ii) if two or more of K3, K4, K5 are H, the dominant difficulty is at the application level → mobile computing; (iii) if K2 is H and none of K3–K5 is H, the dominant difficulty is the radio link → mobile communication; (iv) if K2 is H and exactly one of K3–K5 is H, the case is mixed (borderline).

1.3 Predicted classification of the five cases

Case	K1	K2	K3	K4	K5	Predicted class	Main problem
(a) Rugged tablet, work orders over cellular	H	H	M	M	M	Mobile	Mobile communication
(b) Laptop on Ethernet at different offices	M	None	M	M	L	Nomadic	Mobile computing
(c) Fixed wireless router at substation	L	H	L	L	L	Fixed-wireless	Wireless (radio) communication, not mobile
(d) Push-to-talk handset	H	H	M	L	L	Mobile	Mobile communication
(e) Tablet app: offline meter readings, sync later	H	M	H	M	H	Mobile	Mobile computing
(f) My ambiguous case (below)	M	H	M	H	M	Nomadic with mobile traits	Both – tie

Reasons. (a) The tablet moves with the technician and relies on cellular coverage, so cell selection, handover and coverage dominate; the app issues are moderate. (b) The laptop is only mobile between sessions; the problems are re-acquiring an IP address, re-authenticating, finding local services and keeping session state – there is no radio, so it is nomadic computing. (c) Nothing moves, so handover, tracking and context are irrelevant; only radio propagation, capacity and reliability matter, which makes it a wireless problem but not a mobile one. (d) A PTT handset needs coverage, handover and fast floor/channel access; the handset has no real offline logic. (e) The app is designed to work with no link at all; the hard problems are local storage, queuing, conflict-free synchronisation and battery – the radio is used only briefly.

Own ambiguous case (f): van-tethered tablet

A delivery van carries an LTE router; the driver's tablet joins the van's Wi-Fi and is used only while the van is parked at a customer, although the driver may walk about 30 m around the van. The app caches the route, and a GPS trigger shows the next-stop checklist. K1 = M (moves between stops, not while driving), K2 = H (LTE essential), K3 = M, K4 = H (GPS-triggered behaviour), K5 = M. Under my rules this is nomadic (K1 = M), and because K2 = H while exactly one of K3–K5 (K4) is H, rule (iv) marks it as mixed: a genuine tie between communication (LTE backhaul) and computing (context awareness). **Test:** if the driver also uses the tablet while the van is moving, K1 → H and K3 → H, and the case flips to mobile computing; if the app is removed, it becomes a plain nomadic-communication case. This shows the criteria are sensitive to the right variables rather than just to labels.

1.4 Simulation of case (e): send-immediately vs store-and-forward

Model. One slot = 1 minute (assumed). The simulation runs 2,000 slots; the app produces one meter reading every 5 slots (400 updates). The link is a two-state (up/down) Markov process whose long-run availability is $A \in \{90\ \%, 60\ \%, 30\ \%\}$ with a mean outage length of 10 slots (assumed – outages in the field are bursty, not independent). When the link is up it can send up to 5 queued updates per slot. **Strategy 1 (send immediately):** the update is sent at generation time; if the link is down it fails and is lost (no retry). **Strategy 2 (store-and-forward):** every update is queued and sent FIFO when the link is up. **Counted measures:** delivered on time (delay 0), delivered late (delay > 0), lost (never delivered; for store-and-forward this means still queued when the run ends), and mean delivery delay of delivered updates. Fixed seed 42 for the main table; a 200-seed average (seeds 1–200) is given to show the result is not a one-seed artefact.

> **Prediction (made from theory, before running the code):** Immediate delivery ≈ availability (360 / 240 / 120 of 400). Store-and-forward delivers essentially everything. For store-and-forward, an update generated during an outage waits for the residual outage (mean ≈ 10 slots), so mean delay ≈ $(1 - A) \times 10 \approx 1.0$, 4.0 and 7.0 slots.

Table Q1-a – single run (seed 42), 400 updates per run

Link availability	Strategy	Delivered on time	Delivered late	Lost / undelivered	Mean delay (slots)
90%	Send immediately	368	0	32	0.00
	Store-and-forward	367	33	0	0.86
60%	Send immediately	249	0	151	0.00
	Store-and-forward	247	153	0	3.65
30%	Send immediately	111	0	289	0.00

Link availability	Strategy	Delivered on time	Delivered late	Lost / undelivered	Mean delay (slots)
	Store-and-forward	110	289	1	7.76

Table Q1-b – mean of 200 runs (seeds 1–200)

Availability	Immediate: delivered	Immediate: lost	S&F: delivered	S&F: of which late	S&F: undelivered	S&F mean delay	S&F max delay
90%	361.7	38.3	399.7	38.3	0.34	0.92	31.1
60%	241.1	158.9	399.0	159.3	0.99	3.98	45.6
30%	119.8	280.2	398.7	281.4	1.32	7.14	50.6

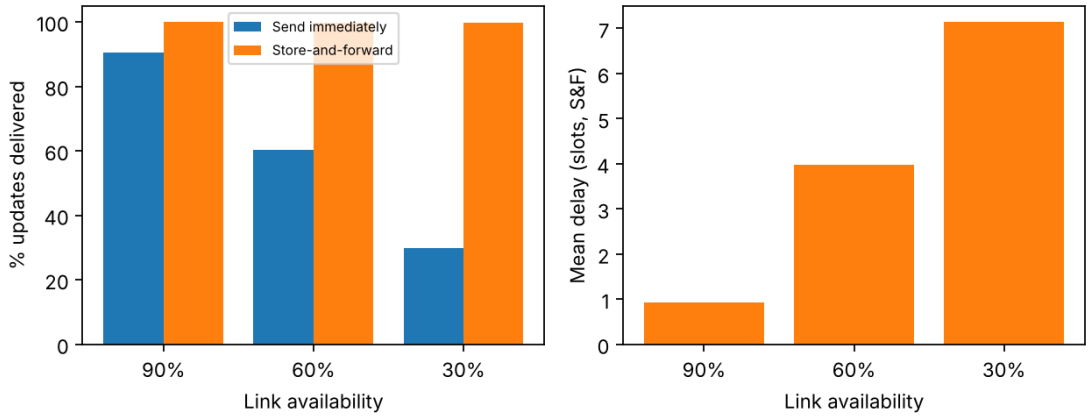


Figure Q1 – % of updates delivered and mean store-and-forward delay (200-run average).

> **Finding:** The prediction is supported. Immediate sending loses exactly the outage fraction (32 / 151 / 289 of 400 in the seed-42 run) while store-and-forward loses 0 / 0 / 1 (the one update still queued at the end). The price is delay: mean 0.9, 4.0 and 7.1 slots (predicted 1.0, 4.0, 7.0), with worst cases of 30–50 slots. At 30 % availability 70 % of updates arrive late, which matters if the control room needs near-real-time data.

1.5 Inquiry

Characteristics of mobile computing exposed that a better wireless link does not solve:

- **Disconnection is a normal state, not an error.** Even at 90 % availability, about 10 % of updates hit an outage and the longest wait in the seed-42 run was 30 slots. A faster radio does not remove outages in tunnels, basements or remote substations; the application must queue, retry and tolerate delay.
- **Data consistency and synchronisation.** Queued updates arrive late and possibly out of order; the server needs timestamps, idempotent updates and conflict rules (for example two readings for the same meter).
- **Resource limits.** The queue needs persistent local storage and the radio needs batching to save battery; at 30 % availability the device may hold hundreds of readings.
- **Context and location.** Readings should be tagged with time and place at capture, not at upload.

Device categories needing application-level changes: the offline meter-reading tablet (e) – local database, queue, sync protocol, conflict handling; the cellular work-order tablet (a) – caching of work orders, resumable downloads, graceful degradation; the PTT handset (d) – jitter buffering,

floor-control timeouts and reconnection logic (a smaller change, because voice is real-time and cannot be queued); the van tablet (f) – cached maps and route. The Ethernet laptop (b) needs only configuration-level changes (DHCP, VPN re-establishment) and the fixed-wireless router (c) needs none at the application level, which is exactly the reasoning for classifying (c) as a wireless problem and (e) as a mobile-computing problem.

Conclusion. “Wireless” and “mobile” are not synonyms, and mobile communication (making the link work while moving) is not the same as mobile computing (making applications work when the link is unreliable, the device is small and the context keeps changing). The simulation shows that even a perfect radio standard would still require a store-and-forward application design.

Q2 – Sharing the Channel: FDMA vs TDMA Under Different Traffic (6 marks)

2.1 Usable capacity after overheads

Option 1 – FDMA. 2 MHz / 25 kHz = 80 nominal channels. I assume one 25 kHz guard channel at each band edge (−2 → 78) and one channel for control/signalling (−1 → **77 traffic channels**). Per-channel in-band synchronisation and signalling are assumed to cost 5 % of capacity, so payload efficiency = 0.95. Payload rate assumed 1 bit/s/Hz × 25 kHz × 0.95 ≈ 23.75 kbit/s per channel.

Option 2 – TDMA (GSM-style). 2 MHz / 200 kHz = 10 carriers nominal. A GSM band keeps about 100 kHz at each edge, so 9 carriers fit → 9 × 8 = 72 slots; two slots are used for broadcast/common control and a signalling channel → **70 traffic slots**. A normal burst lasts 156.25 bit periods: 114 data bits, 26 training-sequence bits, 6 tail bits, 2 stealing flags and 8.25 guard bits, so payload efficiency = 114 / 156.25 = **0.730**. Gross rate per slot = 270.833 kbit/s / 8 = 33.85 kbit/s, payload ≈ 24.7 kbit/s (refs: ETSI/3GPP TS 45.001–45.002; Schiller; Rappaport).

Quantity	FDMA (25 kHz)	TDMA (200 kHz × 8)
Nominal channels	80	80 (10 × 8)
After band-edge guards	78	72 (9 carriers)
After control channels	77	70
Payload efficiency (assumed / from burst format)	0.95	0.730
Payload rate per user channel	≈ 23.75 kbit/s	≈ 24.7 kbit/s
Usable fraction of 80 nominal channels (count only)	96 %	87.5 %
Usable payload channel-equivalents (count × efficiency)	73.2	51.1

Equal nominal capacity therefore does **not** mean equal usable capacity: both lose channels to guards and control, and TDMA additionally loses 27 % of every slot to training, tail and guard bits. Because the per-user payload rates are similar (≈ 24 kbit/s), the practical user capacity difference is mainly the channel count (77 vs 70). All utilisation figures below are measured against each scheme's own usable gross capacity; "nominal" utilisation is measured against the 80 nominal channels.

2.2 Predictions

> **Prediction (made from theory, before running the code):** (i) Continuous voice: both schemes show high payload utilisation because a dedicated channel is fully used; FDMA is slightly better (more channels, less overhead). (ii) Bursty data at about 10 % duty: both are poor – a dedicated channel is idle 90 % of the time, so payload utilisation should be ≤ 10 % × efficiency for both, and the two schemes should be almost indistinguishable because the dominant loss is the idle time, not the framing. (iii) 70/30 voice–data mix: mean duty ≈ 0.7 × 1 + 0.3 × 0.1 = 0.73, so utilisation ≈ 0.73 × efficiency × occupancy ≈ 55 % for FDMA and ≈ 45 % for TDMA.

2.3 Traffic datasets and simulation

Dedicated-channel allocation is simulated as an event-driven loss system: a call that finds all channels busy is blocked; otherwise it holds one channel for its whole duration plus a set-up time (assumed 0.2 s FDMA, 0.6 s TDMA – TDMA must also acquire timing advance and slot synchronisation). Horizon 7,200 s, seed 7. **Payload utilisation** = Σ(duty × call time × efficiency) /

(channels × horizon); **wasted capacity** = channel-time that is busy but not carrying payload (idle during gaps + framing overhead + set-up).

Dataset	Design	Why it is contrasting
D1 Continuous voice	Poisson arrivals, 65 Erlang (mean hold 120 s exponential), duty = 1.0, 3,858 calls	Baseline; dedicated allocation is a perfect fit
D2 Bursty data	Same arrivals, duty = 0.1	Identical offered sessions but only 10 % of the time is active
D3 70/30 mix	Same arrival process, 70 % voice (duty 1.0) and 30 % data (duty 0.1)	Realistic mix; tests intermediate behaviour
D4 Edge case	Synchronised arrivals: 100 very short calls (mean 3.5 s incl. 0.5 s minimum) start within the same second every 30 s, 70/30 mix; 24,000 calls	Average load is low but the arrival pattern is hostile – tests blocking independent of utilisation

Table Q2 – results (counted from simulation; seed 7)

Dataset	Scheme	Channels	Blocking	Channel occupancy	Payload utilisation (usable)	Payload utilisation (80 nominal)	Wasted capacity
D1 Voice	FDMA	77	1.7 %	80.9 %	76.7 %	73.8 %	4.2 %
D1 Voice	TDMA	70	5.1 %	86.0 %	62.5 %	54.7 %	23.6 %
D2 Bursty	FDMA	77	1.7 %	80.9 %	7.7 %	7.4 %	73.2 %
D2 Bursty	TDMA	70	5.1 %	86.0 %	6.2 %	5.5 %	79.8 %
D3 Mix 70/30	FDMA	77	1.0 %	80.4 %	54.1 %	52.1 %	26.3 %
D3 Mix 70/30	TDMA	70	4.0 %	86.1 %	44.3 %	38.7 %	41.8 %
D4 Edge	FDMA	77	21.7 %	12.5 %	8.2 %	7.9 %	4.4 %
D4 Edge	TDMA	70	30.0 %	13.7 %	6.2 %	5.4 %	7.5 %

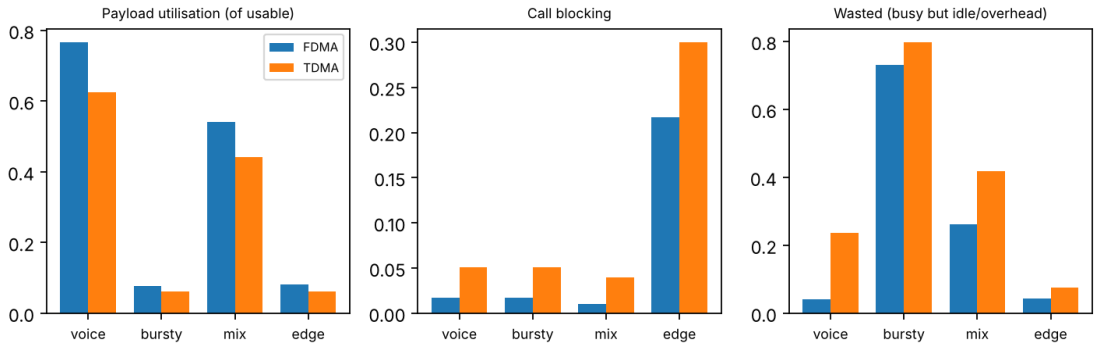


Figure Q2 – payload utilisation, blocking and wasted capacity for the four datasets.

Finding: (i) Voice: FDMA 76.7 % vs TDMA 62.5 % payload utilisation and 1.7 % vs 5.1 % blocking – predicted ordering confirmed. (ii) Bursty: 7.7 % vs 6.3 %, with 73 % and 80 % of usable capacity wasted although occupancy is 81–86 % – both are terrible, as predicted. (iii) Mix: 54.1 % vs 44.3 % (predicted ≈ 55 % and ≈ 45 %).

2.4 Inquiry – where the schemes diverge and why

- **Overhead per channel:** TDMA spends 27 % of each slot on training/tail/guard bits (voice: 62.5 % vs 76.7 % payload utilisation, 23.6 % vs 4.2 % wasted).
- **Channel count:** 70 vs 77 channels at the same 65 Erlang gives blocking 5.1 % vs 1.7 % (voice) and 4.0 % vs 1.0 % (mix); the Erlang-B curve is steep near full load, so a 9 % loss in channels triples blocking.

- **Set-up cost matters only for short calls (D4):** TDMA's extra 0.4 s of synchronisation held the channel longer, raising blocking from 21.7 % to 30.0 %. Note that occupancy was only 13 % – blocking came from the synchronised arrival burst (100 calls in one second against 70–77 channels), not from average load.
- **Where they do NOT diverge:** under bursty traffic (D2) the dominant waste is idle channel time caused by dedicated allocation. Both schemes waste 73–80 %, so changing from FDMA to TDMA (or back) cannot fix it; the allocation policy must change (this is the basis for Q7).

2.5 Design decision – campus telemetry site

A report is 200 B payload + 20 B header (assumed) = 1760 bits. At the FDMA payload rate of 23.75 kbit/s one report occupies 74 ms, once per 60 s: duty cycle = 0.12 %. A dedicated channel would therefore sit idle 99.88 % of the time, and the site could serve only 77 (FDMA) or 70 (TDMA) devices at all, whereas a single shared channel under slotted ALOHA (maximum throughput 0.368) can serve about 0.368 × 60 s / 0.074 s ≈ 298 devices, i.e. roughly 23 thousand devices over 77 channels. (A scheduled periodic TDMA slot per device would support ≈ 60 / 0.074 ≈ 810 devices per channel.)

> **Finding:** Dedicated FDMA/TDMA allocation is **not justified** for the 90 % telemetry devices. Use a shared, contention-based or scheduled-periodic access channel for them and keep dedicated channels only for the 10 % of devices with continuous or delay-critical traffic (voice, video). Defence: the Q2 simulation already shows that a 10 % duty cycle wastes 73–80 % of capacity; telemetry at 0.12 % duty cycle wastes 99.9 %, and the dedicated schemes cap the number of devices at the channel count, not at the traffic volume.

Q3 – CDMA: Orthogonality, Spreading and the Near-Far Problem (7 marks)

3.1 Design of the DS-CDMA uplink simulation

- **Spreading.** Each user k has a data bit $b_k \in \{+1, -1\}$ (BPSK) and a code $c_k \in \{\pm 1\}^L$. The transmitted chip stream is $A_k \cdot b_k \cdot c_k$, i.e. every bit is replaced by L chips, so the chip rate is L times the bit rate.
- **Channel.** The base station receives $r = \Sigma_k A_k \cdot b_k \cdot c_k$ (shifted by τ_k chips in the asynchronous cases) + n, with AWGN of variance $\sigma^2 = L / (2 \cdot Eb/N0)$ per chip. I use Eb/N0 = 10 dB, for which the single-user BER is $Q(\sqrt{20}) \approx 3.9 \times 10^{-6}$, so any errors seen are caused by multiple-access interference (MAI), not noise.
- **Despreading and decision.** For desired user 0, $y = \Sigma_i r_i \cdot c_{0,i}$ (correlation over the L chips); decision = sign(y). With no interference $y = L \cdot b_0$ + noise.
- **Processing gain** G_p = chip rate / bit rate = L: 8 → 9.03 dB, 16 → 12.04 dB, 64 → 18.06 dB.
- **Codes.** Walsh–Hadamard rows of order L (orthogonal when aligned) and random ±1 codes (non-orthogonal; normalised cross-correlation ρ has rms ≈ $1/\sqrt{L}$ = 0.354, 0.250, 0.125).
- **Conditions tested:** (1) Walsh, synchronous, equal power; (2) random codes, synchronous, equal power; (3) Walsh, synchronous, one interferer at 10× power (amplitude $\sqrt{10}$); (4) Walsh, asynchronous (random whole-chip offsets, user 0 fixed at offset 0), equal power; (5) Walsh, asynchronous, one interferer at 10× power. Conditions (4)–(5) are my addition to test the "why orthogonal codes still fail" question.
- **Experiment size.** Code lengths L = 8, 16, 64; for each point 10 independent realisations (new data, codes/offsets and noise) × 4,000 bits = **40,000 bits per point**; fixed seeds ($1000 \cdot L + 10 \cdot K$ + realisation index). BER is measured for the desired (weak, equal-power) user 0.

3.2 Predictions (before execution)

Prediction (made from theory, before running the code): (i) Walsh synchronous: all $K \le L$ codes are exactly orthogonal, so MAI = 0 and BER stays at the single-user floor ($\approx 4 \times 10^{-6}$ → expected 0–1 errors in 40,000 bits) independent of K. (ii) Random codes: MAI variance ≈ $(K-1)/L$ so SIR ≈ $L/(K-1)$; BER ≈ $Q(\sqrt{(L/(K-1))})$ rises quickly with K (e.g. L = 8, K = 4: ≈ 5 %; K = 8: ≈ 14 %), and a longer code postpones the rise. (iii) Orthogonal codes with a 10× user: orthogonality does not depend on amplitude, so with perfect chip synchronisation there should be **no** degradation; but when users are not aligned the cross-correlation is no longer zero and the strong user's leakage is multiplied by $\sqrt{10}$, so I predict a large degradation in the asynchronous 10× case, especially for few users and short codes (where one strong user dominates the total interference).

3.3 Results

Table Q3-a – bit errors out of 40,000 for the desired user (Eb/N0 = 10 dB). "–" = not defined (more users than orthogonal codes).

L	Users K	Walsh, sync	Random, sync	Walsh, sync, 1 user 10×	Walsh, async	Walsh, async, 1 user 10×
8	1	0	0	0	0	0
8	2	1	231	1	5	849
8	4	0	1468	0	216	2387
8	6	1	4291	1	2038	3227

L	Users K	Walsh, sync	Random, sync	Walsh, sync, 1 user 10×	Walsh, async	Walsh, async, 1 user 10×
8	8	0	7245	0	3365	4726
8	10	–	6982	–	–	–
8	12	–	8930	–	–	–
16	1	0	0	0	0	0
16	2	0	4	0	0	10
16	4	0	794	0	4	47
16	8	0	2824	0	188	459
16	12	0	4615	0	1767	1992
16	16	1	5575	1	2258	2502
16	20	–	7104	–	–	–
16	24	–	7752	–	–	–
64	1	0	1	0	0	0
64	4	0	4	0	0	1
64	8	0	142	0	0	0
64	16	1	1309	1	6	6
64	32	1	3616	1	270	292
64	48	2	4823	2	1654	1676
64	64	0	6306	0	2882	2915
64	80	–	7071	–	–	–
64	96	–	8778	–	–	–

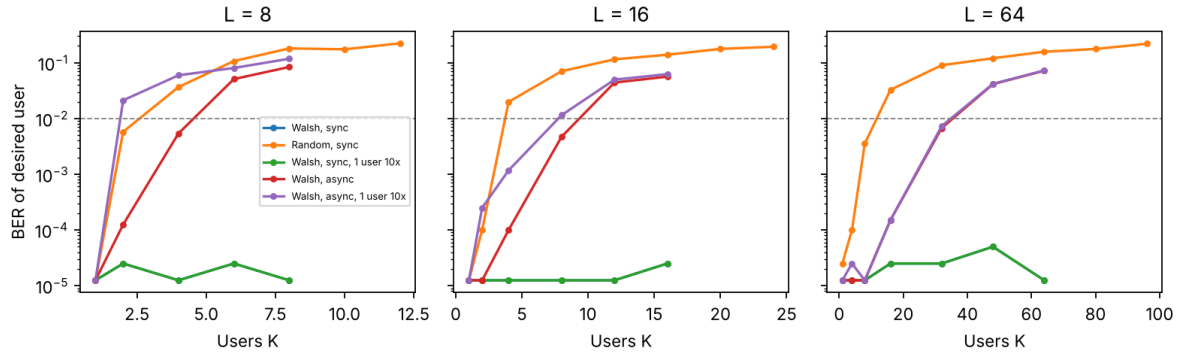


Figure Q3 – BER of the desired user versus number of users for L = 8, 16, 64 (dashed line = BER target 10^{-2}).

Table Q3-b – measured rms cross-correlation contributed per interferer (normalised by L; K = 4) and processing gain

L	Processing gain	Walsh sync	Random sync (theory 1/√L)	Walsh async	Walsh async + 10× user
8	8 (9.03 dB)	0.000	0.309 (0.354)	0.179	0.341
16	16 (12.04 dB)	0.000	0.229 (0.250)	0.094	0.138
64	64 (18.06 dB)	0.000	0.102 (0.125)	0.021	0.029

Table Q3-c – largest K meeting my BER target (BER ≤ 10^{-2} at Eb/N0 = 10 dB, all smaller K also meeting it)

L	Walsh sync	Random sync	Walsh sync + 10×	Walsh async	Walsh async + 10×
8	8	2	8	4	1
16	16	2	16	8	4

L	Walsh sync	Random sync	Walsh sync + 10×	Walsh async	Walsh async + 10×
64	64	8	64	32	32

Finding: (i) Confirmed: synchronous Walsh gives 0–2 errors in 40,000 bits for every K up to L (the 1–2 isolated errors are compatible with the 3.9×10^{-6} noise floor, expected 0.16 per point). (ii) Confirmed in shape: random codes at L = 8 give 1,468 errors (3.7 %) for K = 4 and 7,245 (18 %) for K = 8; at L = 64 the BER stays below 10^{-2} only up to K = 8, i.e. longer codes help but only about K ≈ L/8 users are supported. (iii) Half confirmed: with synchronous chips the 10× user changes nothing (identical error counts to the equal-power case), exactly as predicted; but with asynchronous chips the 10× user is very harmful for short codes – for L = 8, K = 2 the errors rise from 5 to 849 (×170) and for L = 16, K = 4 from 4 to 47. At L = 64 the single strong user is diluted among many other interferers (K = 32: 292 vs 270 errors), so its marginal harm is small; my prediction that the damage would be "especially for short codes and few users" was therefore the right one.

3.4 Inquiry

Why do orthogonal codes still fail when users are unsynchronised or powers are unequal? Walsh codes are orthogonal only at zero relative shift: $\Sigma\, c_j(i) \cdot c_k(i) = 0$. At a shift of τ chips the cross-correlation is the partial correlation of the two sequences, which for Walsh codes is not small (several Walsh codes have large off-peak correlations); the data transitions of the interferer also fall in the middle of the desired bit. The same happens with multipath, where delayed copies of the other users arrive misaligned. In the asynchronous runs the measured per-interferer cross-correlation was 0.179 (L = 8), 0.094 (L = 16) and 0.021 (L = 64) at K = 4, instead of exactly 0. Power imbalance does not break orthogonality itself, but it multiplies any leakage: the interference amplitude from user j is $A_j \cdot \rho_{jk}$, so a 10× power user ($A = \sqrt{10} \approx 3.2$) has 10× the interference power of an equal-power user – the "near-far" effect. Our synchronous 10× test shows that imbalance is harmless only while orthogonality is perfect.

Why is power control essential? The SIR for one user is approximately $SIR \approx G_p \cdot P_0 / \Sigma_j P_j$ (plus noise), so every received power above the desired user's counts directly against the processing gain. A handset next to the base station at 10× the power of a far handset consumes the interference budget of 10 equal users; the far user's SIR drops by 10 dB, which no code length can recover without sacrificing rate. Power control makes all received powers equal at the base station, so capacity becomes limited by the number of users, not by the nearest one. In the table, the asynchronous L = 8 results (users supported 4 vs 1) show how a single un-controlled user collapses the link.

Number of users supported (BER target 10^{-2} at Eb/N0 = 10 dB, my definition). From Table Q3-c: perfectly synchronous Walsh supports K = L (8, 16, 64); asynchronous Walsh with equal power supports about L/2 (4, 8, 32); random codes support only about L/4 to L/8 (2, 2, 8). One uncontrolled 10× user reduces the asynchronous result to 1 (L = 8) and 4 (L = 16) users and has little effect at L = 64 (32). Rule of thumb from the simulation: $K_{max} \approx L/2$ for aligned-within-a-chip-or-two Walsh uplinks and ≈ L/8 for random codes; the processing gain determines capacity.

Limitation: one integer-chip offset per user, no multipath, no coding, 10 code-set realisations per point; BER values below about 10^{-4} are not resolvable with 40,000 bits.

Q4 – OFDMA: Giving Each User the Subcarriers They Can Use Best (6 marks)

4.1 Subcarrier spacing, cyclic prefix and numerical orthogonality

Spacing and symbol duration. Subcarriers are orthogonal over one useful symbol duration T_u when their spacing is Δf = 1/T_u: $\int_0^{Tu} e^{j2\pi k\Delta f\, t} e^{-j2\pi l\Delta f\, t}\, dt = 0$ for k ≠ l. I use Δf = 15 kHz (the LTE value, 3GPP TS 36.211), hence T_u = 66.67 µs, N = 64 subcarriers, total bandwidth 0.96 MHz and sample period 1.04 µs. A narrower spacing means longer symbols (better against delay spread, worse against Doppler); a wider spacing means shorter symbols.

Why a cyclic prefix. A multipath channel spreads each symbol by up to τ_max; the tail of symbol n then overlaps symbol n+1 (inter-symbol interference) and the truncated subcarriers are no longer orthogonal (inter-carrier interference). Copying the last T_cp ≥ τ_max of the symbol in front of it makes the linear convolution with the channel appear circular, so after removing the CP each subcarrier is simply multiplied by its own channel gain H_k and one-tap equalisation works. Cost: efficiency T_u / (T_u + T_cp), e.g. 64 / 80 = 80 % for a 16-sample CP.

Numerical verification (3 subcarriers k = 3, 4, 5). Normalised inner products over one T_u (1,024-sample numerical integration), magnitude:

	k = 3	**k = 4**	**k = 5**
k = 3	1.000	≈ 0 (1.4e-16)	≈ 0 (1.5e-16)
k = 4	≈ 0 (1.4e-16)	1.000	≈ 0 (1.7e-16)
k = 5	≈ 0 (1.5e-16)	≈ 0 (1.7e-16)	1.000

For contrast, a non-integer spacing (subcarriers 3.5 and 4) gives $|\langle\cdot,\cdot\rangle| = 0.637$ – strongly non-orthogonal. An FFT check (IFFT of a unit on bin k, FFT back) leaves a maximum leakage of 5.9e-17 into the other bins. **Cyclic-prefix test:** two consecutive QPSK OFDM symbols through a 4-tap channel (delay spread 3 samples) and one-tap equalisation: error-vector magnitude without CP = 0.065 (6.5 %, caused by ISI/ICI), with a 16-sample CP = 3.0e-16 (zero to machine precision).

4.2 Allocation approaches and predictions

Setup. 8 users, 64 subcarriers. Frequency-selective Rayleigh fading per user: 8 taps with exponential power-delay profile, FFT to 64 subcarrier gains $|H_{uk}|^2$ with mean 1; **1,000 channel realisations**, seed 7. Users sit at different distances, so their average SNRs are 30, 26, 22, 18, 14, 10, 6 and 2 dB (assumed). Rate on subcarrier k for user u is $\log_2(1 + SNR_u \cdot |H_{uk}|^2)$ bit/subcarrier/symbol (Shannon, ideal adaptive modulation); a user's throughput is the sum over its subcarriers (bit per OFDM symbol); ×15 kHz gives kbit/s (CP ignored). Fairness = Jain's index $J = (\Sigma x)^2 / (n \cdot \Sigma x^2)$ of the per-user mean throughputs (J = 1 is perfectly fair, 1/n is the worst).

- **Fixed contiguous blocks:** user u gets subcarriers 8u … 8u+7.
- **Interleaved round-robin:** subcarrier k goes to user k mod 8.
- **Channel-aware best-subcarrier:** each subcarrier goes to the user with the highest rate on it.
- **Design-decision variant (best-subcarrier with minimum-rate guarantee, R_min = 6 bit/symbol):** phase 1 – repeatedly give the weakest unsatisfied user its best free subcarrier until every user reaches R_min; phase 2 – assign the remaining subcarriers to their best user.

Prediction (made from theory, before running the code): Contiguous and interleaved: equal sum throughput (≈ 310 bit/symbol), because without channel knowledge each user's subcarriers are a random sample of the channel; the user SNR spread fixes J ≈ 0.77. Interleaving should only have a small advantage in outages (frequency diversity: a deep fade covering part of a block hurts the block user more). Best-subcarrier: sum throughput roughly double (selection of the largest of 8 rates on each subcarrier, dominated by near users), but J very low (≈ 0.2) and the weakest users almost starved.

4.3 Results

Table Q4-a – sum throughput and fairness (1,000 realisations)

Scheme	Sum (bit/symbol)	Sum (kbit/s)	Jain (long-term)	Jain (mean per realisation)	Outage: user-realisations below R_min = 6
Contiguous blocks	309.8	4647	0.770	0.736	5.3 %
Interleaved RR	309.7	4646	0.768	0.758	1.6 %
Best-subcarrier	619.1	9287	0.228	0.216	65.6 %
Best + min-rate (Rmin=6)	560.3	8404	0.273	0.259	0.0 %

Table Q4-b – mean throughput per user (bit/OFDM symbol); U1 = nearest (30 dB), U8 = farthest (2 dB)

Scheme	U1	U2	U3	U4	U5	U6	U7	U8
Contiguous blocks	73.1	62.5	51.5	41.7	33.0	23.2	15.5	9.4
Interleaved RR	73.0	62.4	52.0	42.0	32.3	23.4	15.4	9.2
Best-subcarrier	433.0	148.2	33.3	4.5	0.2	0.0	0.0	0.0
Best + min-rate (Rmin=6)	354.7	127.2	34.6	10.9	8.8	9.3	7.6	7.1

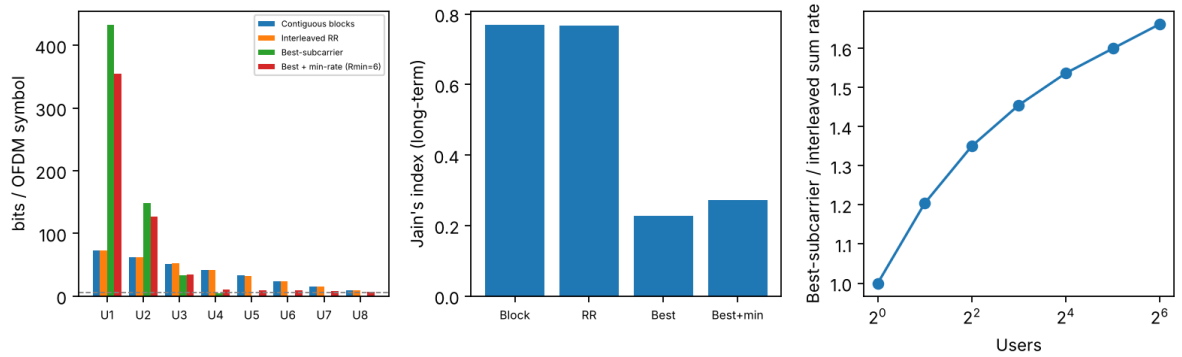


Figure Q4 – per-user throughput (left), Jain's index (middle), and multi-user diversity gain versus number of users (right, equal 15 dB average SNR).

Finding: Predictions supported. Best-subcarrier doubles the sum rate (619 vs 310 bit/symbol, +100 %) but J falls from 0.77 to 0.23: U1 receives 433 bit/symbol while U5–U8 receive 0.2, 0, 0 and 0 (U4 gets 4.5) – effectively starved – and 66 % of all user-realisations fall below R_min. Contiguous and interleaved have the same sum (309.8 vs 309.7); interleaving reduces outages from 5.3 % to 1.6 %, which confirms the frequency-diversity argument. The minimum-rate variant keeps 90 % of the best-subcarrier throughput (560 bit/symbol, +81 % over interleaved) with 0 % outage, but its fairness is still low (J = 0.27) because it only guarantees a floor.

4.4 Inquiry

Why can maximising total throughput starve some users? The scheduler maximises $\Sigma \log_2(1 + SNR_u \cdot |H|^2)$. Because the average SNR of U1 is 28 dB above U8's, U1's rate on a subcarrier exceeds U8's on almost every subcarrier, even when U1 is in a fade; U8 wins a subcarrier only when its fading peak overcomes a 28 dB deficit, which is extremely rare. The policy is optimal for the sum but indifferent to the distribution, so user-level starvation is the expected outcome of maximum-sum allocation when average SNRs differ.

How does the number of users change the benefit of channel-aware allocation? With equal average SNR (15 dB) the ratio of best-subcarrier to interleaved sum throughput was:

Users	1	2	4	8	16	32	64
Gain over interleaved	1.00×	1.20×	1.35×	1.45×	1.54×	1.60×	1.66×

More users give more independent fading draws per subcarrier, so the maximum improves (multi-user diversity), but the gain grows only roughly logarithmically (doubling from 32 to 64 users adds ~4 %) and needs channel-state feedback for U × 64 subcarriers per symbol. With equal average SNRs, long-term fairness is not a problem by symmetry (each user wins equally often in the long run), whereas with unequal SNRs (as in the main experiment) the benefit goes mainly to the strongest users.

4.5 Design decision – lecture hall

Minimum rate chosen: R_min = 6 bit per OFDM symbol per student ≈ 90 kbit/s at 15 kHz. Assumed justification: enough for lecture audio, slide streaming and chat for every student; 8 × 6 = 48 bit/symbol is only 15 % of the 310 bit/symbol that even the simple schemes carry, so the guarantee is cheap. (If video to every student is required, R_min must be scaled up and feasibility rechecked.)

> **Finding:** Select **channel-aware allocation with minimum-rate pre-allocation**. It is the only scheme with 0 % outage against R_min (contiguous 5.3 %, interleaved 1.6 %, pure best-subcarrier 65.6 %) and it still delivers 560 bit/symbol, 81 % more than interleaved. Trade-offs: needs per-subcarrier channel feedback and a scheduler each symbol; the weakest users receive only 7–11 bit/symbol, little above the floor, and J is just 0.27. If lecture-hall policy requires equal service, replace phase 2 by a proportional-fair rule (not simulated here), at the cost of some of the extra throughput. If channel feedback is unavailable, interleaved round-robin is the fallback (1.6 % outage).

Q5 – SDMA: Reusing the Same Channel in Different Directions (6 marks)

5.1 Array-factor derivation

Take N isotropic elements on a line at $x_n = n \cdot d$ ($n = 0 \ldots N-1$), a plane wave arriving at angle θ from broadside, and wavenumber $k = 2\pi/\lambda$. Relative to element 0, element n receives the phase $n \cdot k \cdot d \cdot \sin\theta$. To steer the beam to θ_o the weights are $w_n = (1/N) \cdot e^{-j \cdot n \cdot k \cdot d \cdot \sin\theta_o}$. The array factor is

$$AF(\theta) = \Sigma_n w_n e^{j n k d \sin\theta} = (1/N) \Sigma_{n=0}^{N-1} e^{j n \psi}, \quad \psi = k d (\sin\theta - \sin\theta_o) = (2\pi d/\lambda)(\sin\theta - \sin\theta_o).$$

The sum is a geometric series: $AF = (1/N) \cdot (1 - e^{jN\psi}) / (1 - e^{j\psi}) = (1/N) \cdot e^{j(N-1)\psi/2} \cdot \sin(N\psi/2) / \sin(\psi/2)$, so

$$|AF(\theta)| = | \sin(N\psi/2) / (N \sin(\psi/2)) |, \quad \text{maximum} = 1 \text{ at } \psi = 0 \ (\theta = \theta_o).$$

- **Nulls:** $N\psi/2 = m\pi$, $m = 1, 2, \ldots$ (not multiples of N). For $d = \lambda/2$ and $\theta_o = 0$ the first null is at $\sin\theta = 2/N$, so the null-to-null beamwidth is $2 \cdot \text{asin}(2/N) \approx 4/N$ rad (N = 16: 14.4°).
- **−3 dB beamwidth** $\approx 0.886 \cdot \lambda / (N \cdot d \cdot \cos\theta_o) = 1.77/(N \cos\theta_o)$ rad for $d = \lambda/2$: it narrows as 1/N and broadens by $1/\cos\theta_o$ when steered off broadside.
- **Side lobes:** the first side lobe peaks near $N\psi/2 \approx 3\pi/2$, where $|AF| \approx 2/(3\pi) \approx 0.212$, i.e. **−13.3 dB for large N** – independent of N for uniform weights (tapering is needed to lower it, at the cost of a wider main lobe). $d = \lambda/2$ avoids grating lobes.

> **Prediction (made from theory, before running the code):** Main-lobe width $\propto 1/N$ (halving each time N doubles); first side lobe about −11 to −13 dB for every $N \geq 4$ (it does not improve with N). SIR threshold: 15 dB (my definition, adequate for a QPSK-type link with coding), i.e. $|AF(\theta_B)| \leq 0.178$. Because the side-lobe peaks (−13.3 dB, 0.215) lie above this level, a uniform array can meet 15 dB over **all** larger separations only where the side-lobe envelope $1/(N \cdot \sin(\psi/2))$ falls below 0.178; that gives a minimum separation of about 13° for N = 16 and about 30° for N = 8 (upper bounds), and never for N = 2 or 4.

5.2 Computed patterns and SIR

A is at $\theta_A = 0°$ (beam steered at A), B at $\theta_B = \Delta$ with equal received power; **SIR at A = $|AF(\theta_A)|^2 / |AF(\theta_B)|^2 = -20 \cdot \log_{10}|AF(\Delta)|$**. Angles scanned at 0.1° resolution; table values at 5° steps. Omni: SIR = 0 dB at every Δ. 120° sector (assumed ideal: flat inside, −25 dB outside the sector): SIR = 0 dB for any two users inside the same sector.

Table Q5-a – pattern metrics (d = λ/2)

N	−3 dB beamwidth @ 0°	Null-to-null @ 0°	Highest side lobe @ 0°	Theory: 1.77/N rad (°)	−3 dB beamwidth @ 45°
2	60.0°	≈ 180° (no null inside ±90°)	none (null only at endfire)	50.8°	78.0°
4	26.3°	60.0°	-11.3 dB	25.4°	40.6°
8	12.8°	28.9°	-12.8 dB	12.7°	18.4°
16	6.4°	14.3°	-13.1 dB	6.3°	9.0°

Table Q5-b – SIR (dB) at user A, beam steered at A (θ_A = 0°), B at separation Δ; ideal pointing.
Values below 15 dB (threshold) are not acceptable.

Δ (°)	N = 2	N = 4	N = 8	N = 16
5	0.1	0.4	1.8	8.6
10	0.3	1.7	8.4	13.2
15	0.7	4.0	29.1	29.2
20	1.3	7.8	13.0	20.9
25	2.1	14.4	15.5	20.5
30	3.0	≥ 60 (null)	≥ 60 (null)	≥ 60 (null)
35	4.1	16.9	17.9	22.3
40	5.5	12.7	16.8	29.9
45	7.1	11.4	22.9	24.2
50	8.9	11.5	31.4	31.6
55	11.0	12.5	20.0	23.8
60	13.6	14.4	17.9	36.9

Table Q5-c – same, with 2° rms angular spread / pointing error on both users (power-averaged over 4,000 draws)

Δ (°)	N = 2	N = 4	N = 8	N = 16
5	0.1	0.4	1.6	5.2
10	0.3	1.7	7.3	14.0
15	0.7	3.9	17.5	18.9
20	1.3	7.5	13.4	19.9
25	2.1	13.6	15.3	21.2
30	3.0	23.4	23.7	24.7
35	4.1	16.6	18.0	22.9
40	5.5	12.8	17.0	26.0
45	7.0	11.4	21.9	24.2
50	8.8	11.5	26.9	27.5
55	11.0	12.5	19.9	23.8
60	13.5	14.3	17.9	30.0

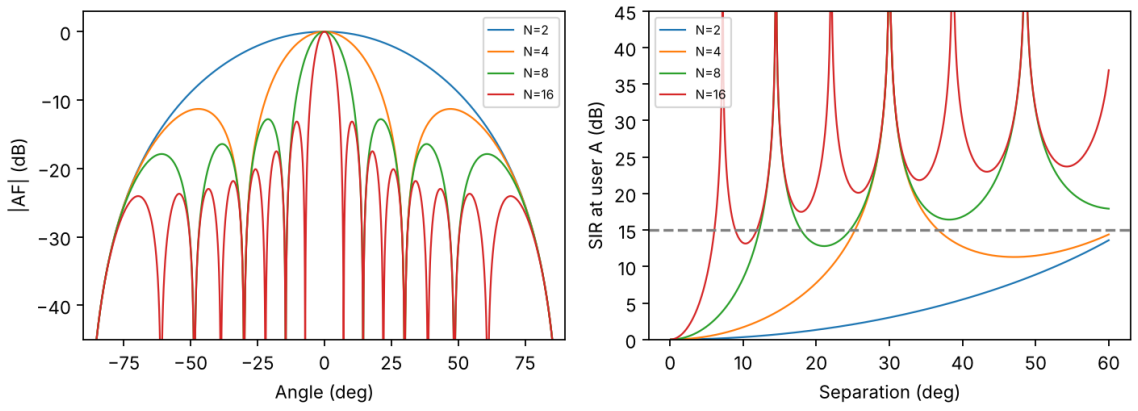


Figure Q5 – array patterns for N = 2, 4, 8, 16 (left) and SIR at A versus separation (right, dashed = 15 dB threshold).

Table Q5-d – minimum separation meeting SIR ≥ 15 dB. "First" = smallest Δ at which the threshold is reached at all (possibly only inside a null); "sustained" = smallest Δ beyond which the threshold holds for every larger Δ up to 60°.

N	First (steer 0°)	Sustained (steer 0°)	Sustained (steer 0°, 2° spread)	Sustained (steer 45°)
2	never	never	never	never
4	25.3°	never	never	never
8	12.3°	24.7°	24.8°	21.8°
16	6.1°	11.9°	11.8°	20.9°

Finding: Predictions supported. Beamwidth halves with each doubling of N (−3 dB: 60.0°, 26.3°, 12.8°, 6.4°; theory 1.77/N rad agrees from N = 8 upwards) and the side lobe approaches the −13.3 dB limit (−11.3, −12.8, −13.1 dB) instead of improving. Minimum sustained separation for 15 dB: never for N = 2 and 4, 24.7° for N = 8 (predicted ≤ 30°) and 11.9° for N = 16 (predicted ≤ 13°). Steering to 45° widens the beam by ≈ 1/cos45° (N = 16: 6.4° → 9.0°) and almost doubles the required separation (11.9° → 20.9°). A 2° angular spread barely changes the large-N result but removes the deep nulls that gave isolated good separations (for example at 30° for N = 4 and 8), so null positions must never be relied on in practice.

5.3 Inquiry

What limits SDMA in practice? (1) **Angular spread:** scattering around the user makes the arrival direction a fan, not a line, so beams cannot be narrower than the spread and nulls fill in (Table Q5-c). (2) **Clustered users:** the minimum separation grows with the beamwidth, not with the user count. (3) **Side lobes:** uniform arrays are stuck near −13 dB, so a nearby strong user (near-far) overwhelms the beam's rejection – a 10 dB stronger user at 20 dB rejection leaves only 10 dB SIR. (4) **Steering loss:** the beam broadens with $1/\cos\theta_o$ at wide angles. (5) Cost and complexity: N RF chains, calibration, direction-of-arrival estimation, mutual coupling, and tracking of moving vehicles.

Why can two users in nearly the same direction not be separated? Each user's signal arrives with a steering vector a(θ); the array can separate users only if their steering vectors are distinguishable. For θ_A ≈ θ_B the vectors are almost parallel, the normalised correlation is $\rho = a(\theta_A)^H a(\theta_B)/N = AF(\theta_B)$ (the same quantity as the pattern value) → 1, and any weight vector that nulls B also nulls most of A. Zero-forcing separation costs a noise-enhancement penalty of $1/(1 - |\rho|^2)$: for N = 16 it is 0.6 dB at Δ = 5°, 6.4 dB at 2° and 12.1 dB at 1°, i.e. the gain vanishes just when the users are closest. Only more elements (narrower beams), larger aperture or a different dimension (time, frequency, code) can separate them.

5.4 Design decision – toll plaza

Geometry: lane width 3.5 m (assumed), 6 lanes, the antenna on a gantry at range d. Angular separation between adjacent lanes = atan(3.5/d), across all six lanes = atan(17.5/d); vehicles queued in the same lane differ only in range, i.e. 0° apart.

Range d (m)	Adjacent lanes	Across 6 lanes
30	6.7°	30.3°
60	3.3°	16.3°
100	2.0°	9.9°
200	1.0°	5.0°
400	0.5°	2.5°

Finding: **Do not rely on SDMA as the primary access method at the toll plaza.** A N = 16 array needs ≈ 12° sustained (≥ 6° at best) separation, but adjacent lanes are only 2–7° apart at 30–100 m and vehicles in the same lane are at 0°. Most vehicles arrive in the same direction, which is the clustered-user case in which the array factor gives $|\rho| \approx 1$ and the noise-enhancement penalty is already 6 dB at 2° and 12 dB at 1° (N = 16). Use TDMA/OFDMA scheduling (time/frequency separation, small payloads and short sessions) and use the antenna array only as a supplement: a sector or beam to cover the gate, interference nulling toward neighbouring cells, and SDMA reuse between the plaza and far-away cells.

Q6 – Choosing Multiple-Access Techniques for Real Deployments (7 marks)

6.1 Criteria and weights

Six criteria were used (the ratings of each technique do not depend on the deployment; the weights do). Ratings are 1 (poor) to 5 (excellent) and are traced to my Q2–Q5 measurements.

Criterion	Evidence-based rating logic (own results)
C1 Capacity under mixed traffic	OFDMA 5: best-subcarrier gives +100 % sum rate over interleaved (Q4). CDMA 4: processing gain and K = L Walsh users with 0 errors when synchronised (Q3). SDMA 3: only when users are separable (Q5). TDMA 3, FDMA 3: dedicated allocation gives 44–54 % payload utilisation on the 70/30 mix (Q2).
C2 Efficiency for bursty / small reports	Dedicated TDMA/FDMA ≤ 8 % utilisation at 10 % duty and 0.1 % at telemetry duty (Q2): rating 2. CDMA 3 and OFDMA 3: capacity can be shared dynamically but only with access/scheduling overhead (Q7 shows ≈ 3 messages per burst). SDMA 2.
C3 Interference / jam robustness	CDMA 5: processing gain 9–18 dB spreads a narrowband jammer (Q3). SDMA 3: beams/nulls reject directional interference only if separable (Q5). TDMA, FDMA, OFDMA 2: a narrowband jammer kills a carrier/subcarrier set.
C4 Frequency-selective fading / high rate	OFDMA 5: each subcarrier sees flat fading, CP removes ISI (Q4). CDMA 4: rake receiver combines multipath. FDMA 3 (narrowband flat fading, whole channel lost in a fade), SDMA 3, TDMA 2 (200 kHz needs an equaliser).
C5 Low device complexity / control overhead	FDMA 5 (filters only), TDMA 4 (frame sync), OFDMA 2 (FFT, tight sync, scheduler, channel feedback), CDMA 2 (power control is mandatory, Q3), SDMA 2 (array, calibration, DOA).
C6 Spatial reuse	SDMA 5 (Q5), CDMA 2 and OFDMA 2 (frequency reuse 1 possible but interference-limited), TDMA 1, FDMA 1.

Rating matrix (1–5)

Technique	C1	C2	C3	C4	C5	C6
SDMA	3	2	3	3	2	5
TDMA	3	2	2	2	4	1
FDMA	3	1	2	3	5	1
CDMA	4	3	5	4	2	2
OFDMA	5	3	2	5	2	2

Weights per deployment (sum = 1) and justification

Deployment	C1	C2	C3	C4	C5	C6	Why
D1 Wide-area cellular voice	0.30	0.05	0.15	0.10	0.20	0.20	Capacity (0.30) and cost/complexity of millions of handsets (0.20) dominate; reuse (0.20) because cells must be re-used across a wide area; voice is continuous so bursty efficiency is only 0.05.
D2 Jam-resistant tactical link	0.10	0.05	0.50	0.10	0.10	0.15	Robustness to interference/jamming is the mission requirement (0.50); capacity, bursty efficiency and complexity are secondary.
D3 High-rate stadium downlink	0.30	0.10	0.05	0.30	0.10	0.15	Spectral efficiency (0.30) and robustness to frequency-selective, multipath-rich fading at high rate (0.30) matter most; reuse (0.15) between sectors.
D4 5,000 low-rate sensors	0.05	0.35	0.10	0.05	0.35	0.10	Small, bursty reports (0.35) and cheap, low-power devices (0.35); capacity per Hz and jam robustness matter little for a sensor cell.

6.2 Scores, choices and sensitivity

Deployment	SDMA	TDMA	FDMA	CDMA	OFDMA	Top	Runner-up	Margin
D1 Wide-area cellular voice	3.15	2.50	2.75	3.30	3.25	CDMA	OFDMA	0.05
D2 Jam-resistant tactical link	3.15	2.15	2.30	3.95	2.65	CDMA	SDMA	0.80
D3 High-rate stadium downlink	3.10	2.35	2.65	3.45	3.90	OFDMA	CDMA	0.45
D4 5,000 low-rate sensors	2.50	2.65	2.70	2.85	2.65	CDMA	FDMA	0.15

Sensitivity check. Each weight was multiplied by 0.8 and by 1.2 in turn (all other weights unchanged and the vector renormalised to 1): 12 perturbations per deployment.

Deployment	Choice changes?	Minimum winner–runner-up margin over all 12 perturbations
D1 Wide-area cellular voice	YES – C1 +20 % → OFDMA; C3 −20 % → OFDMA	0.009
D2 Jam-resistant tactical link	No	0.667
D3 High-rate stadium downlink	No	0.415
D4 5,000 low-rate sensors	YES – C5 +20 % → FDMA	0.011

6.3 Primary technique and hybrid chosen

Deployment	Primary	Hybrid / reasoning
(1) Wide-area cellular voice	CDMA (score 3.30)	Primary CDMA with FDMA carriers and SDMA sectorisation (reuse). OFDMA (3.25) is a statistical tie – the choice is not robust. Real systems have used both (IS-95 vs GSM/LTE).
(2) Jam-resistant tactical link	CDMA (3.95)	Direct-sequence CDMA plus FDMA frequency hopping (spreading across carriers) and SDMA null steering toward jammers. Margin 0.80: robust.
(3) High-rate stadium downlink	OFDMA (3.90)	OFDMA with SDMA beams per sector (stadium has many users in separable directions) and TDMA scheduling of frames. Margin 0.45: robust.
(4) 5,000 low-rate sensors	Narrowband FDMA channels with slotted random access (hybrid), chosen over the raw score winner CDMA (2.85 vs FDMA 2.70)	The score lead of CDMA vanishes when the complexity weight rises by 20 % (FDMA wins) – the decision flips, so I choose the technique that wins in both cases: cheap FDMA nodes with contention-based access, based on the Q2 telemetry result (0.12 % duty cycle) and the Q3 power-control burden for 5,000 cheap nodes.

6.4 Inquiry

Why do real systems combine techniques? Each technique shares one physical dimension – frequency (FDMA), time (TDMA), code (CDMA), subcarriers (OFDMA) or space (SDMA) – and each has a weakness the others compensate for. GSM divides 25 MHz into 124 FDMA carriers (isolation, simple filters) and each carrier into 8 TDMA slots (dynamic sharing and fine granularity), then reuses frequencies in space across cells (SDMA-style reuse). LTE combines FDMA-like subcarriers (OFDMA) with TDMA scheduling in 1 ms subframes and MIMO/beamforming. My own results explain why: dedicated FDMA/TDMA wastes idle capacity (Q2), CDMA needs power control (Q3), OFDMA starves weak users unless scheduled (Q4), and SDMA fails for clustered users (Q5) – so a single dimension is never enough.

Least robust choice: deployment (1), wide-area voice (margin 0.05; flips to OFDMA with C1 +20 % or C3 −20 %), and deployment (4), sensors (margin 0.15; flips to FDMA when C5 +20 %). Both depend on subjective weights; the criterion weights and ratings are my own judgements, backed by Q2–Q5

simulations only where those simulations measured the quantity (capacity, bursty efficiency, power-control need, fading, spatial separation); jam robustness (C3) and complexity (C5) were not simulated and rest on cited design knowledge.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 marks)

Selected limitation: dedicated FDMA/TDMA channels sit idle under bursty traffic (evidence from Q2 datasets D2/D3 and the telemetry decision). I do not replace the scheme with a "more advanced standard"; I diagnose where the capacity goes and change the allocation policy using information the system already has (the terminal's buffer/activity state).

A. Diagnose the bottleneck (2 marks)

Experiment used for the diagnosis (the original solution). Pool of C = 70 slots (the TDMA usable slots from Q2). Sessions arrive as a Poisson process (mean length 120 s). Each session alternates bursts (mean 1 s) and gaps with a 10 % duty cycle (time unit = 1 tick = 0.1 s, 30,000 ticks, 3,000 warm-up, seed 11). **Original policy:** a session that finds a free slot keeps it for its whole life (request + grant + release = 3 messages); otherwise it is blocked.

Offered sessions (Erlang)	Slot-ticks held	Slot-ticks carrying data	Utilisation of held slots	Session blocking	Carried load (% of 70 slots)
50	1,329,470	132,871	10.0 %	0.0 %	7.0 %
100	1,843,522	183,378	9.9 %	35.5 %	9.7 %
200	1,876,638	187,985	10.0 %	66.7 %	9.9 %
300	1,882,415	187,870	10.0 %	76.9 %	9.9 %

- **Where the capacity is spent:** on holding slots. Slot-ticks held rise from 1.33 M to 1.88 M and then saturate at 70 × 27,000 = 1.89 M, while slot-ticks that carry data stay between 133 k and 188 k.
- **Which counted quantity grows:** blocked sessions – 0 → 823 → 3,004 → 5,071 as load increases (0 %, 35.5 %, 66.7 %, 76.9 %) – although the carried load never exceeds about 10 % of the slots.
- **Idle, wasted or repeated?** Idle: exactly 90 % of every held slot-tick is empty (utilisation 10.0 % at every load, equal to the duty cycle). Nothing is repeated; signalling is tiny (≈ 0.24 messages per burst).
- **Root cause in the design:** the allocation granularity is the **session** (minutes) while the traffic granularity is the **burst** (≈ 1 s). The slot is committed at set-up, using no information about activity, and is never returned during the gaps, so the capacity limit is set by the number of sessions, not by the amount of traffic.

B. Design objective (2 marks)

At the same C = 70 slots, the same offered sessions and the same burst traffic (10 % duty), the redesign must achieve **all** of:

1. Utilisation of allocated slots ≥ 70 % (original 10 %).
2. Session blocking ≤ 1 % at 200 Erlang of offered sessions (original 66.7 %).
3. Mean burst access delay ≤ 1 tick (0.1 s) and signalling ≤ 5 messages per burst (the original's 0.24 per burst is the price of its simplicity).

C. Revised strategy (4 marks)

What changes and what it exploits. The slot is allocated **per burst**, not per session. The property exploited is the on/off structure of the traffic and a piece of information the terminal already has – its transmit buffer state. A session is admitted without a slot; the slot is requested when a burst starts and returned shortly after it ends. Resource holding during gaps is therefore avoided.

- **Admission control (replaces channel-count blocking):** admit a session while the expected active load fits: Σ duty ≤ 0.8·C, i.e. at most S_max = 0.8 × 70 / 0.1 = 560 sessions.
- **Reservation channel:** 16 request mini-slots per tick. A terminal with a new burst sends a request in a random mini-slot; one sender per mini-slot succeeds. To avoid ALOHA instability the base station broadcasts an access probability p = min(1, 16 / backlog).
- **Grant:** FIFO queue; the base station grants a free slot to the head of the queue.
- **Hold timer (T_h = 2 ticks):** after the last burst data the slot is kept for 2 ticks in case the next burst follows closely, then released with an explicit message. This avoids a request/release cycle for back-to-back bursts.
- **New costs:** per-session state and a request queue at the base station; request, grant and release messages (≈ 3 per burst); contention collisions; a small access delay; and admission control that must estimate the duty cycle. The bandwidth of the 16 mini-slots is not deducted from the 70 slots in my model (limitation).

Pseudocode

```
on session_arrival:
    if active_sessions < S_max: admit (no slot yet) else block
each tick, for the base station:
    backlog = sessions with burst waiting and no request accepted
    broadcast p = min(1, 16/backlog)
    for each waiting terminal (prob p): send request in random mini-slot m
    for each mini-slot with exactly 1 request: enqueue terminal in FIFO
    while FIFO not empty and free_slots > 0:
        grant slot to FIFO.head; terminal transmits
    for each held slot whose burst has ended:
        hold_counter += 1
        if hold_counter > T_h: release slot (explicit release message)
on terminal new_burst:
    if it still holds its slot: transmit at once (no request)
    else: become "waiting" (request on next tick)
```

Message sequence for one burst:

Step	Terminal → Base station	Base station → Terminal
1 burst starts, no slot	REQUEST (random mini-slot)	–
2 if collision	retry on a later tick with probability p	access probability p broadcast each tick
3 slot free	–	GRANT (slot number)
4 burst transmitted	data in granted slot	–
5 burst ends + T_h ticks with no new data	RELEASE	slot returned to the pool

D. Prediction before implementing (1 mark)

Prediction (made from theory, before running the code): (1) Utilisation of allocated slots ≈ burst length / (burst length + hold) = 10 / (10 + 2) ≈ 83 %, against 10 % for the original. (2) Session blocking falls to ≈ 0 up to the admission limit: the active demand is about 0.1 × sessions, so 200–300 Erlang need only 20–30 of 70 slots; carried load should rise ≈ 4× at 300 Erlang. (3) Mean access delay below 0.5 tick because the reservation channel (16 mini-slots, ≈ 5.9 successful requests/tick at its maximum) exceeds the request rate (≈ 3.3 per tick at 300 Erlang). (4) Signalling ≈ 3 messages

per burst – about 12× the original, plus collisions. (5) At duty ≈ 0.95 (continuous voice) the redesign should bring no benefit and cost extra signalling, so the original should be at least as good.

E. Implementation and validation (2 marks)

Both policies are simulated with the same code, parameters, ticks, burst model (mean 10 ticks) and seed (11); the only difference is the allocation policy. Counted measures are given in Table Q7-a (10 % duty) and Table Q7-b (other duty cycles, 60 Erlang).

Table Q7-a – 10 % duty cycle, C = 70 slots

Offered (Erlang)	Policy	Session blocking	Util. of held slots	Carried load (% of slots)	Mean access delay (ticks)	Messages per burst	Request collisions
50	Original	0.0 %	10.0 %	7.0 %	0.000	0.25	0
50	Redesign	0.0 %	83.6 %	7.4 %	0.034	2.97	485
100	Original	35.5 %	9.9 %	9.7 %	0.000	0.24	0
100	Redesign	0.0 %	83.6 %	14.6 %	0.070	3.01	1,966
200	Original	66.7 %	10.0 %	9.9 %	0.000	0.24	0
200	Redesign	0.0 %	83.6 %	28.4 %	0.161	3.10	8,716
300	Original	76.9 %	10.0 %	9.9 %	0.000	0.24	0
300	Redesign	0.0 %	83.6 %	42.9 %	0.291	3.23	23,894

Table Q7-b – other duty cycles at 60 Erlang

Duty cycle	Policy	Session blocking	Util. of held slots	Carried load	Mean access delay (ticks)	Messages per burst	Slot-ticks held
0.4	Original	1.9 %	41.7 %	35.2 %	0.000	0.061	1,594,218
0.4	Redesign	0.0 %	84.9 %	32.4 %	0.143	2.765	721,990
0.95	Original	3.8 %	100.0 %	86.8 %	0.000	0.024	1,641,041
0.95	Redesign	0.0 %	100.0 %	85.8 %	0.038	0.051	1,622,488

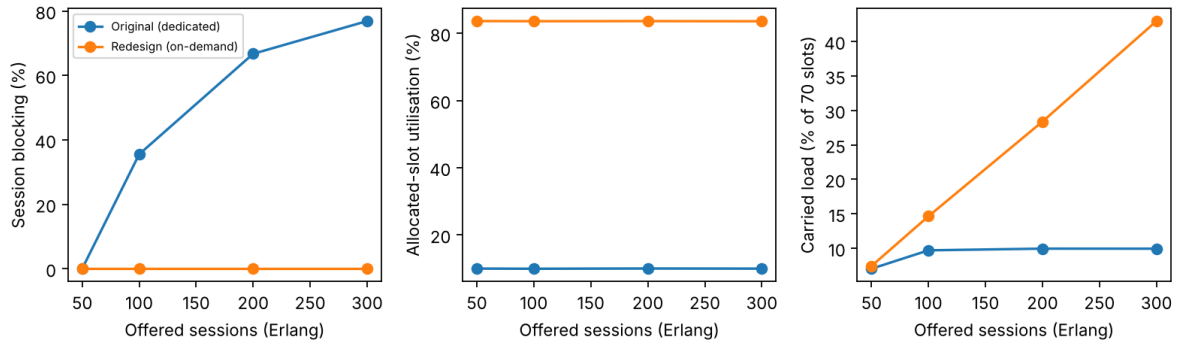


Figure Q7 – session blocking, utilisation of held slots and carried load: original vs redesign (10 % duty).

F. Defend the final design (2 marks)

Finding: Prediction supported, with one correction. (1) Utilisation of held slots 83.6 % at every load (predicted 83 %; original 10 %). (2) Blocking 0 % at 50–300 Erlang (original 0, 35.5, 66.7, 76.9 %); at 300 Erlang the carried load is 42.9 % vs 9.9 % (4.3×). (3) Mean access delay 0.03–0.29 ticks (3–29 ms), inside the objective of ≤ 1 tick. (4) Signalling 2.97–3.23 messages per burst against 0.24 for the original (≈ 12–13×) – inside the ≤ 5 objective but a real cost; collisions grow from 485 to 23,894 per run between 50 and 300 Erlang. (5) For duty 0.95 both policies hold the slots 100 % full, carried load 87 % vs 86 %, the redesign adds nothing in utilisation and doubles the signalling (0.051

> vs 0.024 messages per burst). The one correction is that the original also rejects 3.8 % of sessions at that load against 0 % for the redesign, because admission is by expected load instead of channel count.

Magnitude of improvement. At the same 70 slots the redesign carried 4.3× the traffic at 300 Erlang and removed all blocking; allocated-slot utilisation rose 8.4× (10 % → 83.6 %). At duty 0.4 (60 Erlang) the redesign held 722 k slot-ticks to carry 613 k data ticks, whereas the original held 1,594 k to carry 664 k (55 % fewer slot-ticks; 92 % of the data), so the freed slots could be given to other users.

New costs and when the original is still preferable: (i) 12× the signalling per burst and a contention channel whose own bandwidth I did not subtract from the 70 slots – a real system must include it; (ii) access delay and jitter (small here but not zero, and unbounded in principle at overload) – unacceptable for hard real-time voice where the original gives zero delay and a deterministic guarantee; (iii) extra state and an admission rule that depends on a good estimate of the duty cycle – if sessions in fact have higher duty than assumed, the 83 % ceiling (and the $S_{max} = 560$) overbooks the slots and bursts queue; (iv) the original is simpler and equal in efficiency when traffic is nearly continuous (duty ≈ 0.95). **Conclusion:** for bursty data and telemetry, per-burst allocation with a short hold timer is justified; for continuous voice keep dedicated slots; a deployment mixing both should run the two policies side by side (voice slots dedicated, data slots on demand). **Limitations:** a single seed per point, a simplified reservation channel, a fixed hold time ($T_h = 2$ ticks, not optimised) and an assumed burst model (geometric mean 10 ticks).

References and files

- ETSI / 3GPP TS 45.001 and TS 45.002 – GSM physical layer on the radio path and multiplexing (carrier spacing 200 kHz, 8 slots, 156.25-bit burst, 114 data bits).
- 3GPP TS 36.211 – LTE physical channels and modulation (15 kHz subcarrier spacing).
- J. Schiller, Mobile Communications, 2nd ed., Pearson, 2003 – multiplexing, GSM, handover, GPRS/HSCSD, DECT.
- T. S. Rappaport, Wireless Communications: Principles and Practice, 2nd ed., Prentice Hall, 2002.
- J. G. Proakis and M. Salehi, Digital Communications, 5th ed., McGraw-Hill, 2008 – direct-sequence spread spectrum and its BER.
- C. A. Balanis, Antenna Theory: Analysis and Design, 4th ed., Wiley, 2016 – uniform linear arrays.
- R. Jain, D.-M. Chiu and W. Hawe, “A Quantitative Measure of Fairness and Discrimination for Resource Allocation in Shared Computer Systems”, DEC-TR-301, 1984 – Jain’s index.
- P. Viswanath, D. Tse and R. Laroia, “Opportunistic beamforming using dumb antennas”, IEEE Trans. Inf. Theory, 2002 – multi-user diversity.

Please verify the citation details against the library copies before submitting.

Files supplied with this report: code/q1_q2.py, q3_cdma.py, q4_ofdma.py, q5_sdma.py, q6_decision.py, q7_redesign.py and README.md; data/ (CSV and JSON for every result); fig/ (all figures).